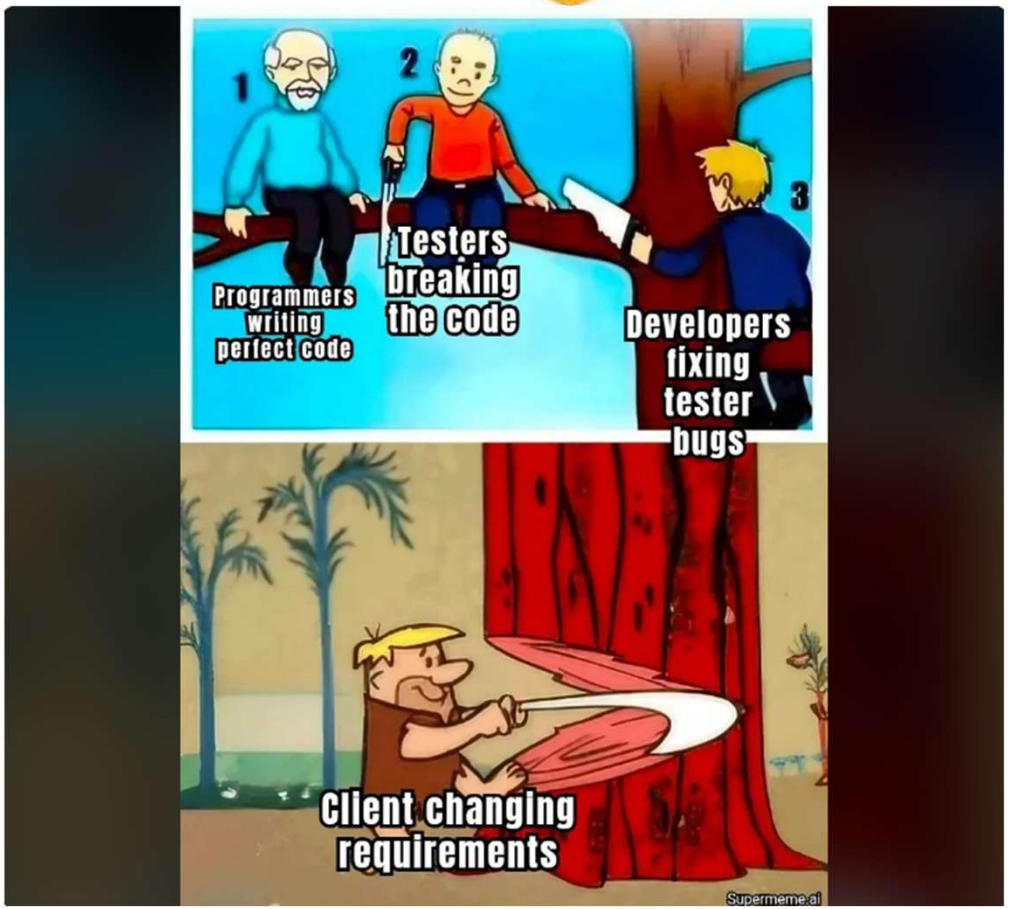
Ursinus
COLLEGE

CS375 Spring 2026

Ralph Rostock

Week 5

Software Architecture: Engineering Structure Under Constraint


1
2
3
Programmers writing perfect code
Testers breaking the code
Developers fixing tester bugs
Client changing requirements
Supermeme.ai

Team Standup Meetings

Last Week

Requirements Validation, Stakeholders, Backlogs, And Planning Realism

Learning Objectives

By end of class students can:

- Define software architecture
- Explain how architecture responds to non-functional requirements
- Compare architectural styles via tradeoffs
- Understand architectural decisions & ADRs
- Identify architectural risk
- Distinguish architecture from detailed design

What Is Software Architecture?

Architecture is the set of significant structural decisions about a system that are costly to change later.

Key Attributes:

- System organization
- Major components
- Interaction boundaries
- Deployment structure
- Technology constraints

Why “Costly to Change Later” Matters

Examples:

- Switching from Monolith → Microservices
- Changing Database Type
- Replacing Synchronous with Event-Driven

Non-Functional Requirements Drive Architecture

Functional Requirements

- Define system capabilities
- Influence feature decomposition

Non-Functional Requirements

- Constrain structure
- Drive tradeoffs
- Usually determine architecture style

Architecture exists primarily to satisfy quality attributes at scale.

Example: “Build a Messaging App”

Start Simple:

- Users can send messages
- Users can receive messages

Now Add Constraints:

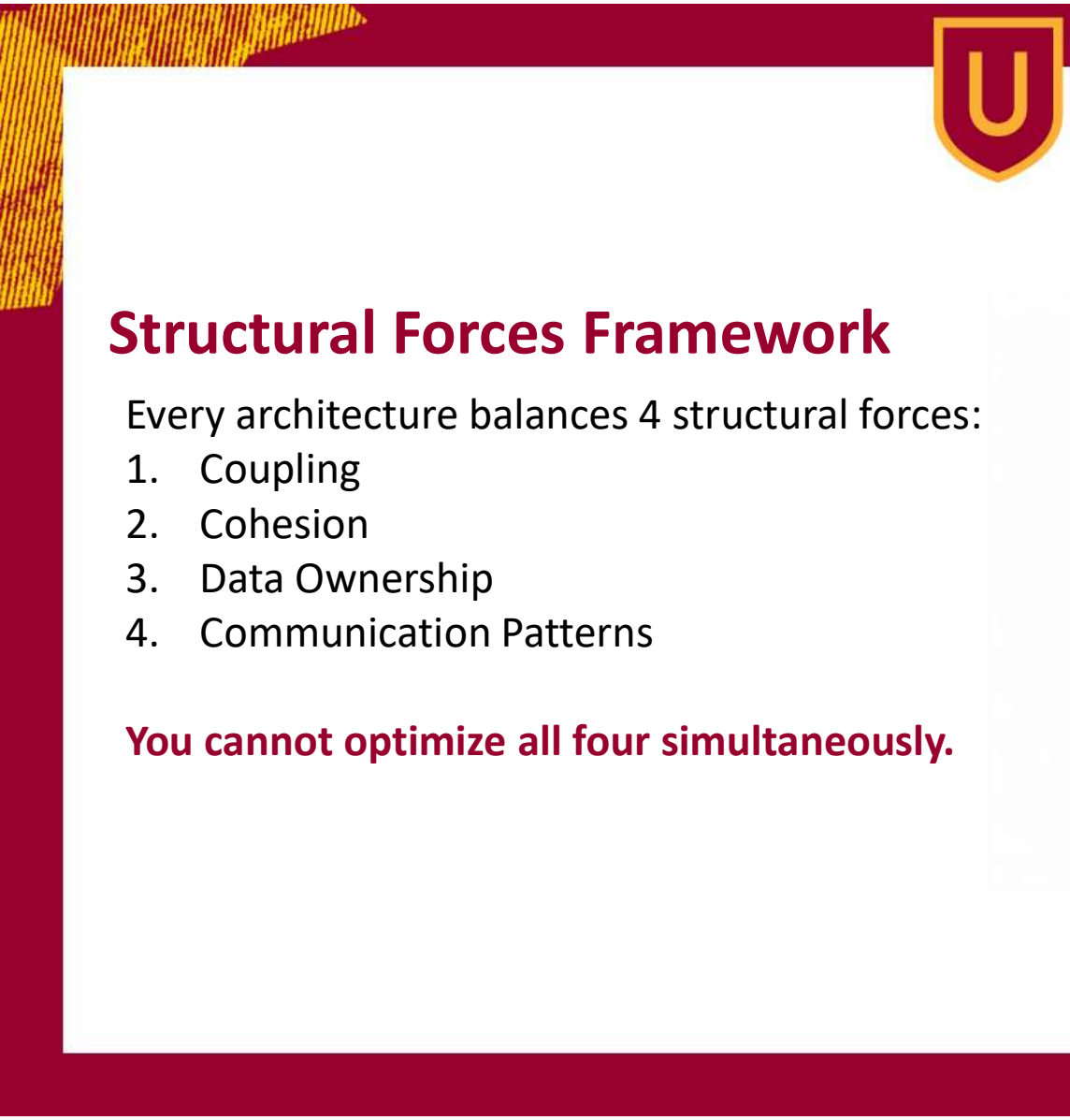
- Must support 10 million users
- 99.99% Uptime
- End-to-end encryption
- Real-time delivery

Quality Attributes Create Structural Tension

- Performance ↔ Maintainability
- Consistency ↔ Availability
- Security ↔ Usability

Improving one often stresses another

Architectural styles are reusable structural solutions to recurring pressures.

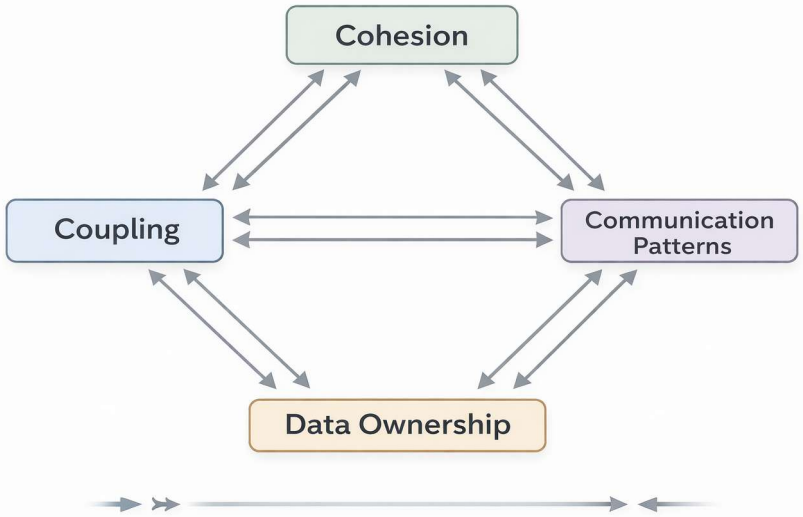


Structural Forces Framework

Every architecture balances 4 structural forces:

1. Coupling
2. Cohesion
3. Data Ownership
4. Communication Patterns

You cannot optimize all four simultaneously.



Layered Architecture

Layered architecture optimizes for maintainability and conceptual clarity.

Structural Force Analysis:

- Coupling: Vertical coupling between layers
- Cohesion: High within layer, low across layers
- Data ownership: Usually centralized
- Communication: Linear (top-down calls)

Monolith

A monolith is a system deployed as a single unit.

Structural Force Analysis:

- Coupling: Potentially high (if unmanaged)
- Cohesion: Can be high or low
- Data ownership: Centralized
- Deployment boundary: Single unit

Microservices

Architectural choice to move structural boundaries to the deployment level.

Structural Force Analysis:

- Coupling: Lower at code level, higher at network level
- Cohesion: High within service
- Data ownership: Decentralized
- Communication: Network-based

Event-Driven Architecture

Decoupled components communicate asynchronously by producing and reacting to state changes—or "events"—in real time.

Structural Force Analysis:

- Coupling: Temporal decoupling
- Cohesion: Event producers unaware of consumers
- Data ownership: Often decentralized
- Communication: Asynchronous

Architecture Comparison

Style	Optimizes For	Complexity Moves To
Monolith	Simplicity	Internal code
Microservices	Team scaling	Distributed system
Event-driven	Scalability	Debugging & consistency
Layered	Clarity	Rigidity

Architectural Style Evolution

Many systems evolve through styles

Typical lifecycle:
Monolith ➡️ Modular Monolith ➡️ Services ➡️ Event-driven components

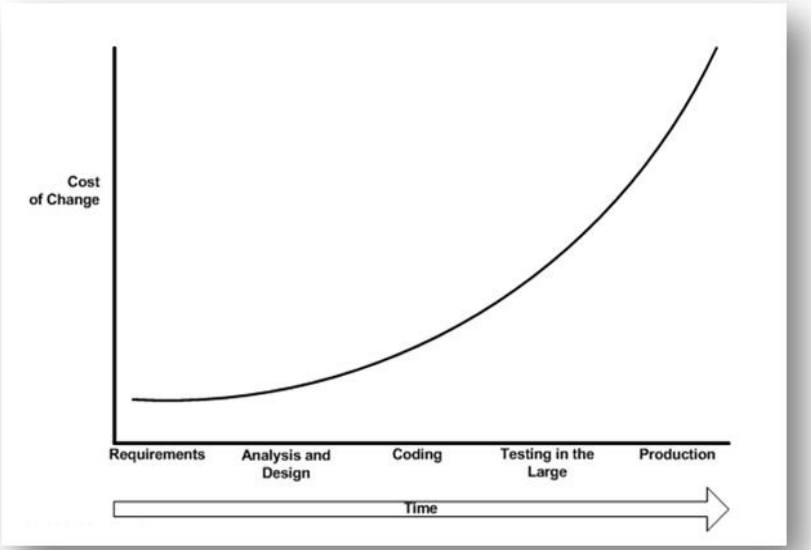
Architecture Is Decision-Making Under Uncertainty

It is not about being right or wrong.
It is about making defensible decisions with incomplete information.

Irreversibility & The Cost of Change

Not All Decisions Are Equal

- Two-way door decisions
  - Easy to reverse
  - Low switching cost
  - Ex: Swapping logging libraries
- One-way door decisions
  - Structural change
  - High switching cost
  - Ex: distributed vs centralized architecture



Overengineering & Premature Scaling (The Risk of Solving Tomorrow's Problems Today)

Every architectural choice introduces:

- Cognitive load
- Operational complexity
- Testing complexity
- Deployment complexity

Microservices, for example, introduce:

- Network failures
- Versioning challenges
- Observability complexity
- Distributed transactions

Architecture as Risk Management

Architecture decisions reduce specific risks:

- Scalability risk
- Availability risk
- Security risk
- Organizational risk

But introduce new risks:

- Operational risk
- Complexity risk
- Cost risk

So, architecture is about choosing which risks you are willing to carry.

Architectural Decision Records (ADR)
(If It's Not Documented, It's Accidental)

ADRs are:

- A record of reasoning
- A protection against "why did we do this"
- A tool for future engineers

ADR Structure

- Context
- Decision
- Alternatives Considered
- Consequences

ADR: A Simple Example

- Context: 4-Person team, 1 semester, unknown future usage
- Decision: Adopt modular monolith architecture
- Alternatives Considered: Microservices; fully serverless architecture
- Consequences
  - Faster development
  - Lower operational complexity
    - Harder to scale beyond moderate growth
    - Requires disciplined modular boundaries

Conway's Law

"Organizations design systems that mirror their communication structure"
- Melvin Conway (1967)

System boundaries reflect team boundaries

Communication Cost Scales Faster Than You Think

Number of communication paths in a team:
n (n-1) / 2

3 people ➡️ 3 paths
5 people ➡️ 10 paths
8 people ➡️ 28 paths

Architecture Can Enable or Constrain Teams

Architecture That Enables:

- Clear ownership boundaries
- Independent deployment
- Well-defined APIs
- Modular codebase

Architecture That Constrains:

- Shared databases across teams
- Tightly coupled modules
- Unclear ownership
- Frequent cross-team coordination required

Architecture Fails Gradually, Then Suddenly

Most systems don’t collapse on day one.
They accumulate structural debt:

- Hidden coupling
- Patch-on-patch fixes
- Workarounds
- Shortcuts under deadline pressure

Then one day:

- New feature takes 3x longer
- Small change breaks multiple areas
- Scaling requires rewrite

That’s architectural erosion

Architecture Failure #1: The Big Ball of Mud

A system with:

- No clear boundaries
- High coupling
- Shared global state
- Unclear ownership

What Cause It?

- No architectural guardrails
- Rushing features
- No module boundaries
- “Just get it working”

Architecture Failure #2: Premature Distribution

Teams adopt:

- Microservices
- Event-driven architecture
- Serverless fragmentation

Before:

- They need scale
- They need independent team velocity

Architecture Failure #3: Ignoring Non-Functional Requirements

Example: System designed purely around features

- No caching
- No replication
- No load balancing
- No failure planning

Then traffic grows

Now:

- Emergency scaling
- Patches under stress
- Expensive rework

Architecture Failure #4: Shared Database Anti-Pattern

Two services

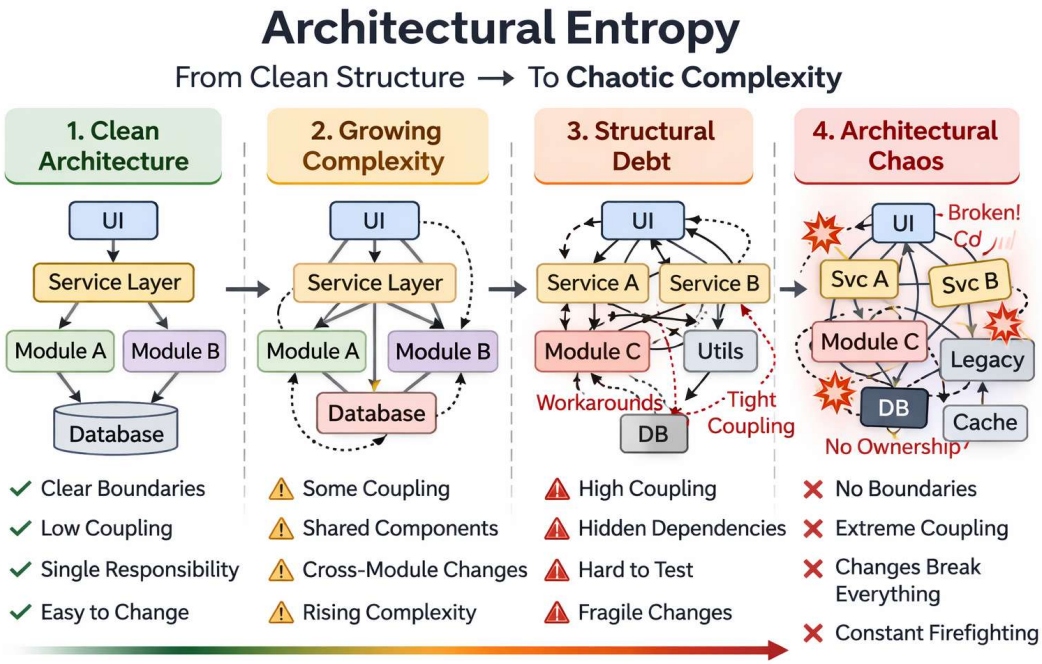
- Both directly access same database tables

Looks simple.

But:

- Tight coupling
- Schema changes break multiple services
- No clear data ownership
- Deployment coordination required

Architectural Entropy: Why Systems Drift



Over time, structure degrades without deliberate effort.

- Quick fixes accumulate
- Ownership boundaries blur
- Dependencies increase
- Design intent fails

Architecture doesn't fail all at once. It erodes.

Architectural Erosion: When Drift Becomes Damage

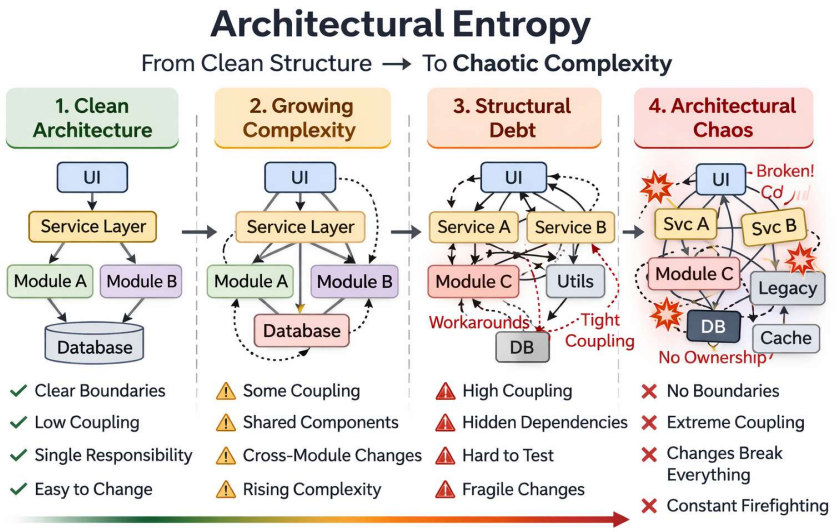
Early Signs:

- Increased coordination required
- Small changes feel risky
- Build/test cycles slow down

Later Symptoms:

- Refactoring becomes scary
- Features take longer than expected
- Developers avoid certain parts of the system

When structure decays, velocity drops.



Architecture is a Set of Decisions

Architecture is the collection of:

- Structural decisions
- Technology decisions
- Boundary decisions
- Dependency decisions
- Constraint decisions

Architecture is the set of decisions that are hard to reverse.

Architecture Evolves With the System

Architecture must adapt to:

- New requirements
- Increased scale
- Team growth
- Changing constraints

Architecture that doesn’t evolve becomes friction.

Architecture vs. Detailed Design

Architecture answers:

- How is the system divided?
- How do major parts interact?
- Where are the boundaries?
- What constraints exist?

Detailed Design answers:

- How does this component work internally?
- What data structures are used?
- What algorithms are implemented?
- How is behavior organized?

Architecture defines structure. Design defines behavior within structure.

Architectural Stewardship (System-Level Responsibility)

Healthy architecture requires:

- Protect boundaries
- Avoid unnecessary coupling
- Evaluate system-level trade-offs

Good architecture is maintained, not declared.

Next Week

Detailed Design & Design Principles

Assignment: Weekly Standup Reflection

https://rrostock1.github.io/Ursinus-CS375/Assignments/StandupReflection

(Individual Assignment)
Due Feb 26 (and every week thereafter)

Assignment: The Overdose

https://rrostock1.github.io/Ursinus-CS375/Assignments/Overdose

(Individual Assignment)
Due Apr 23

Due Next Week (March 5, by Midnight)

- Weekly Standup Reflection